DineFlow · الجزء التقني

شرح السبرنتات 1 و 2 و 3

ما الذي بُني في كل سبرنت، وأين هو في الكود، وكيف تحققنا أنه مكتمل ويعمل.

Sprint 1 · الطاولات و QR | Sprint 2 · المنيو الإلكتروني | Sprint 3 · الطلبات المباشرة

إعداد: أيمن الحواري

2026-10-25

الخلاصة

✓ **السبرنتات الثلاثة مكتملة وموجودة في الكود على فرع `main` .**
كل سبرنت له commit خاص، وكل الملفات الأساسية موجودة في آخر نسخة، وكل اختبارات الـ Backend ناجحة.

3 / 3	241 / 241	81	0
سبرنتات مكتملة	اختبار ناجح حالياً	اختبار أُضيف في السبرنتات 1–3	اختبار فاشل

ما هو DineFlow؟

الضيف يمسح رمز QR الموجود على الطاولة، فيفتح المنيو على جواله، ثم يطلب ويدفع ويقيّم الزيارة. المطعم يرى الطلبات والدفعات والتقييمات مباشرة من لوحة التحكم.

التقنيات المستخدمة

الجزء	التقنية
Backend	`ASP.NET Core 8 Web API`، Clean Architecture، `JWT` ، `SignalR`
قاعدة البيانات	`SQL Server 2022` عن طريق `EF Core 8`
Frontend	`Angular 21` + `Tailwind CSS 4` ، وفيه تطبيقان: `guest` للضيف (عربي وإنجليزي ويدعم RTL) و `merchant` لوحة تحكم المطعم
الاختبارات	`xUnit` ، والـ Services تُختبر على قاعدة `SQLite` في الذاكرة

طبقات الـ Backend

`Domain`
الكيانات وقواعد العمل (مثل Order و Menu و QrToken)

`Application`
الـ Services: حالات الاستخدام مثل إنشاء طلب أو مسح QR

`Infrastructure`
قاعدة البيانات، الـ Migrations، التشفير، تخزين الصور

`Api`
الـ Controllers، الصلاحيات، الـ SignalR Hubs

الاتجاه دائماً من الخارج للداخل: الـ Api يستدعي الـ Application، والـ Application يعتمد على الـ Domain. الـ Domain لا يعرف شيئاً عن قاعدة البيانات.

commit `910ea5e` • **Sprint 1**

المطاعم والطاولات ورموز QR وجلسة الضيف

Epics: `E1` (أساس المطعم والفرع) · `E2` (QR وجلسات الطاولة)

الهدف: نبني الأساس. يكون عندنا مطعم وطاولات، ولكل طاولة QR، وعند مسحه يبدأ الضيف جلسة بدون حساب.

ما تم في الكود

الجزء	ماذا يعمل	الملف
الكيانات	`Tenant` (الشركة)، `Venue` (الفرع)، `DiningArea` (الصالة)، `ServiceLocation` (طاولة، كاونتر، غرفة فندق، مسبح...)، `StaffUser` (الموظف)	`Domain/Venues/`
رمز QR	الرمز نفسه لا يُحفظ في قاعدة البيانات. نحفظ فقط الـ hash الخاص به (`SHA-256`). لو تسرّبت قاعدة البيانات لا يقدر أحد يصنع رموز QR صالحة منها	`QrToken.cs`
توليد الرمز	يُولّد بـ `HMAC-SHA256` وطوله 27 حرف (160 bit). لا يمكن تخمينه وليس رقماً متسلسلاً	`QrTokenService.cs`
جلسة الضيف	`GET /v1/public/qr/{token}` يتحقق من الرمز، ثم ينشئ `GuestSession` ويعطي الضيف `JWT` مؤقت	`GuestService.cs`
عدم كشف الرموز	الرمز غير الموجود والمعطّل والقديم كلها ترجع نفس الخطأ `qr_invalid` ، حتى لا يعرف المهاجم أي الرموز موجودة	`GuestService.cs`
فصل البيانات	كل كيان يطبّق `ITenantOwned` ، وكل استعلام يُفلتر تلقائياً بالـ `TenantId` ، فلا يرى أي مطعم بيانات مطعم آخر (Multi-tenant)	`AppDbContext.cs`
دخول المطعم	`JWT` مع صلاحيات `Merchant` و `MerchantManager` ، وحد لعدد المحاولات (Rate limit) على تسجيل الدخول وعلى مسح QR	`AuthService.cs`
إدارة الطاولات	إضافة صالات وطاولات (فردية أو دفعة واحدة)، وتغيير رمز QR (rotate) أو تعطيله أو تفعيله	`TableService.cs`
الأخطاء	كل خطأ يرجع بصيغة `ProblemDetails` مع `code` ثابت، مثل `venue_unavailable` و `invalid_credentials`	`Api/Common/`

كيف يعمل مسح QR خطوة بخطوة

1. الضيف يمسح الرمز، فيفتح الرابط ويرسل الرمز إلى `/v1/public/qr/{token}` .
2. السيرفر يتأكد أن شكل الرمز صحيح، ثم يحسب الـ hash ويبحث عنه في جدول `QrTokens` .
3. يتأكد أن الرمز فعّال، وأن الطاولة والفرع فعّالين، وأن الشركة غير موقوفة.
4. ينشئ `GuestSession` مربوطة بالطاولة، ويختار اللغة (عربي أو إنجليزي).
5. يرجع للضيف `JWT` مع بيانات المطعم (الاسم، الشعار، اللون، العملة) والطاولة.

واجهة الضيف

صفحة الهبوط بعد المسح، بالعربي والإنجليزي مع RTL، وشاشات للحالات "QR غير صالح" و"المطعم مغلق". الجلسة تُحفظ فلا يحتاج الضيف يمسح مرة ثانية عند التحديث.

لوحة المطعم

تسجيل الدخول، وصفحة **Tables & QR** (إضافة صالات وطاولات، طباعة وتحميل وتغيير الرموز)، وصفحة **Settings** مع معاينة مباشرة لشاشة الضيف.

الاختبارات: 28 اختبار، منها اختبارات تتأكد أن مطعماً لا يرى بيانات مطعم آخر.

commit `3966f0e` • **Sprint 2**

المنيو الإلكتروني والإضافات

Epics: `E4` (أساس المنيو) · `E31` (تحسينات المنيو الإلكتروني)

الهدف: المطعم يبني المنيو من لوحة التحكم، والضيف يتصفحه ويختار الإضافات ويضيف للسلة.

ما تم في الكود

الجزء	ماذا يعمل	الملف
`Menu`	المطعم يقدر يكون عنده أكثر من منيو (مثلاً "All day" و "Breakfast"). لكل منيو أيام وساعات عمل، والساعات تقدر تتجاوز منتصف الليل، وفيه حالة منشور أو مسودة	`Menu.cs`
`MenuCategory`	أقسام المنيو، ومعها مستوى واحد من الأقسام الفرعية	`MenuCategory.cs`
`MenuItem`	الصنف: الاسم بالعربي والإنجليزي، والسعر بالـ minor units (مثلاً 12.500 دينار تُحفظ كرقم صحيح `12500`)، وصورة، وحالة "نفد"، و14 نوع من مسببات الحساسية، وتاغات	`MenuItem.cs`
الإضافات	مجموعات إضافات (`ModifierGroup`)، ولكل مجموعة حد أدنى وحد أقصى للاختيار، واختيارات افتراضية. مثلاً "اختر الحجم" إجباري، و"إضافات" اختيارية	`ModifierGroup.cs`
منيو الضيف	`GET /v1/public/sessions/{id}/menu` يعرض المنيوهات المنشورة، ويحدد هل يمكن الطلب الآن حسب توقيت المطعم المحلي	`GuestMenuService.cs`
إدارة المنيو	إضافة وتعديل وحذف (CRUD) للمنيوهات والأقسام والأصناف، وإعادة الترتيب. زر "نفد" يقدر يستخدمه أي موظف، والباقي للمدير فقط	`MenuAdminService.cs`
رفع الصور	JPEG أو PNG أو WebP بحد أقصى 5MB. نوع الملف يُفحص من محتواه وليس من امتداده. تُخدم من `/media` مع cache طويل	`MenuImageService.cs`
بيانات تجريبية	منيو "All day" و "Breakfast" (07:00–11:30) بالعربي والإنجليزي، مع صنف نافد للتجربة	`DemoMenuSeed.cs`

مثال: كيف يعمل جدول المنيو

منيو "Breakfast" مضبوط على 07:00–11:30. لو فتح الضيف المنيو الساعة 2 ظهراً، يقدر يتصفحه لكنه لا يقدر يطلب منه. السيرفر يحسب هذا بتوقيت المطعم (عمّان) وليس بتوقيت جوال الضيف. ولو كانت الساعات مثلاً 22:00–02:00، فهذا يعني أن المنيو يمتد بعد منتصف الليل.

واجهة الضيف

صفحة المنيو فيها تبديل بين المنيوهات، وبحث، وأقسام، وعرض شبكة أو قائمة، وحالة "نفد". نافذة الصنف تعرض الإضافات الإجبارية والاختيارية مع السعر يتحدث مباشرة، ثم سلة وصفحة مراجعة.

لوحة المطعم

صفحة **Menu** فيها محررات للمنيو والقسم والصنف ومجموعات الإضافات، ورفع صور، وزر "نفد"، وإعادة ترتيب.

الاختبارات: 32 اختبار جديد (المجموع وقتها 60).

commit a5bfed9 • **Sprint 3**

الطلبات المسعّرة من السيرفر وشاشة الطلبات المباشرة

Epics: E6 (السلة والطلب) · E10 (تنبيهات الموظفين)

الهدف: الضيف يرسل الطلب، والسيرفر يحسب السعر بنفسه، والمطبخ والموظفون يرون الطلب فوراً.

ما تم في الكود

الجزء	ماذا يعمل	الملف
حساب السعر	المكان الوحيد الذي يُحسب فيه المجموع. رسوم الخدمة على المجموع الفرعي، والضريبة على (المجموع الفرعي + رسوم الخدمة). يدعم أسعار شاملة الضريبة أو الضريبة مضافة فوق السعر	`OrderPricing.cs`
فحص السلة	السيرفر يسعّر السلة ويُبلغ عن مشكلة كل سطر: صنف نفد أو حُذف، منيو مغلق الآن، إضافات تغيرت أو خارج الحد المسموح	`CartQuoteService.cs`
الطلب	`Order` و `OrderLine` يحفظان نسخة ثابتة من الأسماء والأسعار وقت الطلب، فتعديل المنيو لاحقاً لا يغيّر طلباً قديماً. رقم الطلب يبدأ من 1001 لكل فرع	`Order.cs`
منع التكرار	كل طلب يأتي بـ `clientRequestId`، وعليه Unique index. لو انقطع الإنترنت وأعاد الضيف الإرسال، يرجع نفس الطلب ولا يُنشأ طلب مكرر (Idempotency)	`GuestOrderService.cs`
تغيّر السعر	لو المجموع الذي رآه الضيف يختلف عن حساب السيرفر، يرجع خطأ `409` `price_changed` مع السعر الجديد. لا يُطلب شيء بسعر لم يوافق عليه الضيف	`GuestOrderService.cs`
حالات الطلب	الموظف يغيّر حالة الطلب، وأي انتقال غير مسموح يُرفض (انظر دورة الحياة بالأسفل)	`MerchantOrderService.cs`
إشعارات فورية	`SignalR` : الموظفون يتصلون بـ `/hubs/merchant` ويستقبلون `orderPlaced` و `orderChanged`، والضيف يتصل بـ `/hubs/guest` ويتتبع طلبه مباشرة	`Hubs/OrderHubs.cs`

دورة حياة الطلب

`Submitted` → `Accepted` → `Preparing` → `Ready` → `Completed (Served)`

`Submitted` → `Rejected + reason`

الرفض ممكن فقط والطلب جديد. بعد القبول يقدر الموظف يتخطى خطوات. القيمة `Completed` تظهر في الشاشة باسم "Served".

مثال حساب السعر (السعر شامل الضريبة كما في الأردن)

المجموع الفرعي للأصناف	`JOD 20.000`
رسوم خدمة 10%	`+ JOD 2.000`
المجموع الذي يدفعه الضيف	`JOD 22.000`
منها ضريبة مبيعات 16% (تُعرض ولا تُضاف)	`JOD 3.034 = 22.000 − (22.000 ÷ 1.16)`

واجهة الضيف

صفحة مراجعة الطلب (الأسعار من السيرفر، الرسوم، ملاحظة للمطبخ، زر "Place order" آمن لإعادة المحاولة)، وصفحة تتبع الطلبات مع سبب الرفض إن وُجد.

لوحة المطعم

لوحة **Orders** بأعمدة New و Accepted و Preparing و Ready، مع صوت تنبيه وإشعار منبثق وعدّاد على القائمة وعنوان التبويب، وتمييز الطلب المتأخر. وفي **Settings** صار فيه رسوم الخدمة والضريبة.

الاختبارات: 21 اختبار جديد (المجموع وقتها 81).

قواعد أساسية يلتزم بها الكود

1. **المال لا يُخزّن أبداً كـ float.** نستخدم `long` بالـ minor units مع رمز العملة. الدينار الأردني له 3 خانات عشرية، يعني الدينار الواحد يُخزّن كـ `1000`.
2. **السيرفر هو اللي يسعّر الطلب**، وليس جوال الضيف. الضيف فقط يؤكد السعر الذي حسبه السيرفر.
3. **كل عملية كتابة لها مفتاح منع تكرار** (`clientRequestId`)، فإعادة الإرسال آمنة.
4. **لا نخزّن رموز QR نفسها**، نخزّن الـ hash فقط، والرموز يمكن تغييرها في أي وقت.
5. **فصل بيانات كل مطعم تلقائياً** عن طريق الـ Tenant filter على كل استعلام.
6. **الطلب يحفظ نسخة من الأسعار والأسماء** وقت الطلب، فتغيير المنيو لا يغيّر الطلبات القديمة.

كيف تحققنا أن السبرنتات مكتملة

طريقة التحقق	النتيجة
Git history	كل سبرنت له commit على `main` : `910ea5e` و `3966f0e` و `a5bfed9`
وجود الملفات	كل الملفات الأساسية للسبرنتات الثلاثة موجودة في آخر نسخة ، مثل: `QrToken, GuestService, Menu, ModifierGroup, Order, OrderPricing, CartQuoteService, OrderHubs`
`dotnet test`	241 من 241 اختبار ناجح، صفر فشل
ما لم يُفحص	اختبارات الـ Angular (`npm test`)، والتجربة اليدوية في المتصفح

تشغيل عرض تجريبي للفريق

1. شغّل الـ API (ينشئ قاعدة البيانات ويضيف مطعم تجريبي "Demo Bistro" في عبدون، عمّان):

```
dotnet run --project DineFlow/DineFlow.Api --launch-profile http
```

2. شغّل تطبيقي الضيف والمطعم:

```
npm --prefix web ci
npm --prefix web start
```

3. خطوات العرض:

1. افتح لوحة المطعم على `http://localhost:4300` وسجّل الدخول. بيانات الدخول في `appsettings.Development.json` تحت `Seed:Owner` .
2. **Sprint 1:** ادخل على **Tables** واعرض الطاولات ورموز QR، ثم اضغط **Open** على أي طاولة لتفتح شاشة الضيف.
3. **Sprint 2:** اضغط **View Menu**، واعرض الأقسام والبحث ونافذة الإضافات. ثم في لوحة المطعم علّم صنفاً "نفد" وأعد تحميل منيو الضيف.
4. **Sprint 3:** أضف أصنافاً وأرسل الطلب. سيظهر فوراً في **Orders** مع صوت التنبيه. حرّك الطلب بين الحالات، وشاهد شاشة تتبع الضيف تتحدث مباشرة.